Text to Printable Parts: Which Knowledge Helps an LLM Write, Check and Repair CAD Code?

CAP 6938 (AI Knowledge Systems), Fall 2026 — Final Report

Michael Laager
University of Central Florida
mi740165@ucf.edu

Abstract

3D printers are widely available tools for easily creating 3D objects. Part development, however, still requires knowledge of 3D modelling in CAD software. For this project, we explore how an agentic setup can contribute to the automatic development of these models. The agent takes in a user request as text, determines the scope of the request (an "organic" shape versus a mechanical part) and attempts to generate a valid STL at the correct dimensions. Mechanical parts use CadQuery code; organic shapes either use signed distance functions for sculpting or an optional download from the Thingiverse web service. We show the architectural effects of gating, retrieval, and repair loops on this system. We then compare multiple local and API-driven large language models (LLMs) as the orchestrators. We use dimension bounding and an LLM-as-judge approach on a synthetic dataset of 78 cases ran in multiple configurations (2118 total runs), showing that smaller models exhibit better accuracy when using them. Larger "frontier" models may exhibit improvements, but the agentic workflow has less variance on their overall performance.

1. Introduction

Multiple disciplines make use of 3D generated models: engineers, artists, and additive manufacturers — just to name a few. The industry standard representation of a 3D model is currently the STL format, as it's triangular meshes are easy for supporting software to work with.

Currently, the bar to entry for these disciplines is learning how to use CAD or other modelling software. This issue has been a motivating factor behind agents that produce 3D representations of objects. However, most of these models fall in the text-to-mesh realm, and do not allow for ease of measurement for specifications. Instead, a text-to-code model is able to have deterministic checks ran against the code, allowing easy validation, repeatable results, and parametric exactness.

In addition, a text-to-code model allows more variety of language models to be used for orchestration (multi-modal models are not required). For these orchestrators, there are multiple sources of knowledge that can be used in the aid of mesh structures:

- Retrieved examples from a library (RAG)
- Code feedback
- Mesh bounding box checks
- Visual renders of meshes to check for accuracy
- Model fine-tuning
- Online services or repositories that host user-created models

This leads to the following research questions, which are addressed by this project:

1. How much does the model matter, and is the gap about size or running locally?
2. Do retrieved examples help, and for which models?
3. Can fine-tuning on verified builds close the gap for a small local model?
4. How much do the repair loop and retrying the whole request add?
5. For organic shapes, is sculpting with code better than downloading a model?
6. How far can a visual LLM judge be trusted?

In this project, we present an agent that turns a request into a verified STL. We design a synthetic benchmark of 48 mechanical parts and 30 organic shapes. An visual LLM judge was chosen based on a blind study of 146 user graded parts, which was used to evaluate the rest of the generations. We also provide a training regimen for fine-tuning local models to better suit this task.

2. Related work

Among the project specifications, Text2CAD [9] attempts to follow a similar text-code paradigm. Unlike this project, the output was direct CAD code, not python. A subset of Text2CAD, Text2CadQuery [17] was a subset of

Text2CAD's data, auto transcribed to python by Gemini 2.0 Flash. These queries focused more on abstract geometries without specific dimensions, which translates poorly to our use-case. It also lacks the agentic pipeline of checking resulting geometries for accuracy. CAD-Recode [14] also produces CadQuery code, but from point clouds rather than text. Closest to our visual judge, CADCodeVerify [1] has a vision-language model inspect renders of the generated part and uses its answers to refine the CAD code.

We use a feedback loop similar to Self-Refine [12], Reflexion [15] and Self-Debugging [4], where failed first runs are used to iteratively refine subsequent runs. To increase the initial knowledge of the agent, studies have shown the effectiveness of preloading the agent with relevant knowledge to increase success rate. This is known as retrieval augemented generation (RAG) [11]. Two popular methods have emerged as testing for document relevancy, IDF and embeddings. We will explore this methods in more depth in the system design.

As shown in the LLM-as-a-judge paper [18], using large language models as deciders can yield good results. This method allows for simple grading criteria to be created and ran automatically, saving human time that would otherwise need to be used to evaluate runs.

Finally, to increase the orchestrator's success, methods have been devised to directly influence the output of the tokens. Primarily, QLoRA [6] allows for the fine-tuning of student models from a teacher model. Teacher models, in our case, will be frontier-level LLMs, serving to train small local models.

3. System

We designed an agentic system to take text from a user in a conversational style, then use the resulting specifications to iteratively generate STLs until a passing STL is generated.

3.1. Intake gate

A gate is required to determine the scope of the agentic development. In this system, we define three different generation paths the agent can take: mechanical, organic, or composite. A mechanical part requires exact dimensionality, whereas organic is more fluid in its design. A composite request will combine multiple organic or mechanical requests into a single STL at the end. It is up to the gate model to determine the scope of the user request, and to make clarifications until a valid request can be made.

The initial request by the user, r_0, is given to the *Gate Model*. From this initial request, the agent checks the request for any missing information, like height data or shape. After, the agent drafts a request to be sent along in the pipeline, which the user can then suggest edits or approve.

3.2. Retrieved examples

The agent then gathers similar examples to use as a basis for generating the STL. There are 26 hand-crafted examples, ranging from gears to animals like fish. Examples from Text-to-CadQuery were added initially as more examples, but more examples did not appear to improve performance. Thus, those examples were dropped.

TF-IDF is used as the similarity metric, after initial testing showed embeddings to not improve overall performance. TF-IDF is based on exact word matching, which was shown to be adequate for getting relevant examples.

All of the examples are scored, and the top 3 are appended to the request to allow the builder to see well-formed examples.

3.3. Generate, run, check, repair

The *Builder Model* then gets the resulting output from the *Gate Model*. This model is instructed to build valid CadQuery [2] (Python) code for the requested object. The model also creates a suggested bounding box for the mesh, which is automatically checked against the actual bounding box of the resulting object.

The python code is ran in a sandbox that includes all the necessary libraries for performing generation. If the code fails, the sandbox sends the error message and forces a rerun of the generation. If it executes, it takes the resulting mesh and checks the sizing against the expected bounding box. If those do not match within a percentage threshold, then the agent will also have to try again.

For organic parts, if configured, the model can instead make requests to an online service for parts. In this case, Thingiverse [16] was configured as the online web service. It hosts user-made STLs that can be freely downloaded and used. So, the model makes a search on its API for similar objects. It can either pull the first match, or pull multiple and have the user decide. If a Thingiverse model is selected, the task is considered complete.

3.4. Combined parts

Composite, the third track, first decides what each part requires (mechanical or organic / downloaded). It then runs each of those parts separately in the above desicribed generation loop. After all the parts are gathered, it then composites the STLs into a final shape, merging them with manifold3d [10].

3.5. Model selection and fine-tuning

The agent is designed to work with multiple LLM orchestrators as the backend. Primarily, API driven LLMs were used the most, but it was desirable to create a local LLM that the application could work with.

For the local LLM models, we took the newer Gemma 4 models and Qwen 3.5 and ran those as *Gate Models* and

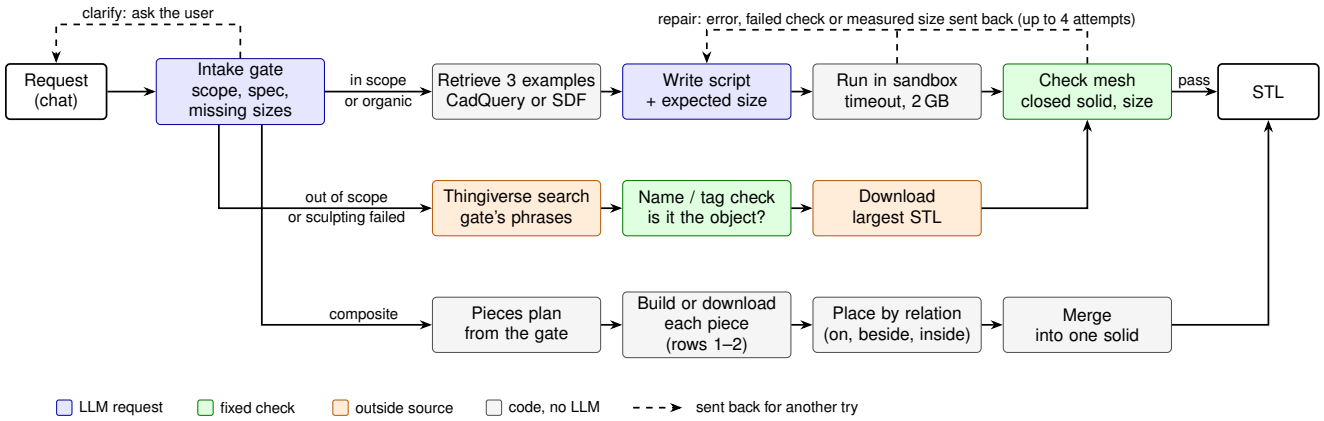


Figure 1. Pipeline. The intake gate (one LLM request) reads the chat and either asks for missing sizes or routes the request. In-scope parts are written as CadQuery code with three retrieved examples; organic objects are sculpted with SDF code through the same loop. Each script runs in a sandbox and the mesh is checked (closed solid, size against the request); a failure is sent back for a repair, up to four attempts. Objects code can't build are searched on Thingiverse, and combined parts are built or downloaded piece by piece, placed and merged.

Builder Models. We also wanted to see the effectiveness of fine-tuning a model specific to this task. Fine tuning, like running these models, was performed locally. Thus, the options available were limited by hardware specifications.

The chosen model for fine-tuning was Qwen 3.5. The fine-tuning was performed by training a 4-bit QLoRA of rank 16 for 2 epochs. It was trained on a generated 1052 teacher conversations: 822 CadQuery, 230 SDF; 990 train / 62 validation; teachers Sonnet and gemini-3.5-flash-lite.

4. Evaluation

4.1. Benchmarks

The total benchmark was made up of 78 requests to the agent. From these 78 requests, 48 were mechanical parts and 30 were organic. Each of the mechanical requests belonged to one of four categories: simple requests (5), medium-difficulty requests (17), hard-difficulty requests (13), and casual requests (13). Casual requests are ones that do not follow a specific mechanical request, similar to organic style requests. Each object in the casual was hand-checked as to whether the model should reject building the part, require building it, or allow either a rejection or build.

The organic suite is similar to the casual requests, except specifically focused on sculpting shapes. For example, the 12th organic benchmark request: "A turtle 60 mm long: a domed shell, four stubby legs, a head poking out the front and a short tail."

4.2. Builders and configurations

Table 1 lists the builders and the configurations each was run with. Claude Sonnet 5.5, Claude Haiku 5.5 and Gemini 3.5 Flash Lite run through their providers' APIs and cover a range of price and strength. Gemma 4 31B runs on Google's servers and Gemma 4 12B, the app's default local model, on our own GPU; as two sizes of one model family, they separate the effect of model size from that of running locally. The two Qwen3.5 4B models, the base model and our fine-tuned version (Sec. 3.5), show how far a small local model gets.

The main suite is run in two configurations. With RAG, the app's setup, the three most similar of the 26 examples are added to the prompt; without RAG, the prompt has no examples. On the organic suite, models write signed distance function (SDF) code in the style of fogleman/sdf [8] instead of CadQuery, with the three most similar of 11 SDF examples in the prompt. In every configuration a run allows up to 4 attempts: each failed attempt sends the error, or the measured size of a part that came out the wrong size, back to the model for a repair. The Thingiverse baseline uses no model: it searches Thingiverse with a short phrase for each organic case and keeps the first result whose name or tags contain the object's name.

Each builder runs every case 3 times per configuration (the Thingiverse search once), for 2,118 runs in all. All runs used one version of the code and took place from November 6 to 8, 2026. Three sets of runs were added after the first results came in: Gemma 4 31B, Haiku, and the Qwen runs without RAG. None of them changed an earlier run.

4.3. Metrics

The first metric on the main suite is *right size*. A run counts as correct when it ends with a valid mesh whose bounding box matches the expected one within 3% or 1 mm per side, whichever is larger, or when it declines a request that should be declined. For the 3 cases marked *either*, building and

		Main suite		Organic
Builder	Runs on	RAG	no RAG	SDF
Claude Sonnet 5.5	Anthropic	✓	✓	
Claude Haiku 5.5	Anthropic	✓	✓	✓
Gemini 3.5 Flash Lite	Google	✓	✓	✓
Gemma 4 31B	Google	✓		
Gemma 4 12B†	local	✓		
Qwen3.5 4B fine-tuned	local	✓	✓	✓
Qwen3.5 4B (base)	local	✓	✓	✓
Thingiverse	Thingiverse		–	download

Table 1. Builders and the configurations each was run with. *RAG*: three retrieved examples in the prompt (the app's setup); *no RAG*: none; *SDF*: organic objects sculpted with SDF code, with three of 11 SDF examples; Thingiverse search takes the first matching download instead. API models run on the provider's servers. Every model run allows up to 4 attempts and is repeated 3 times per case (Thingiverse once); 2,118 runs in all. Local models run in Ollama on one RTX 3080 (10 GB). †The app's default local model.

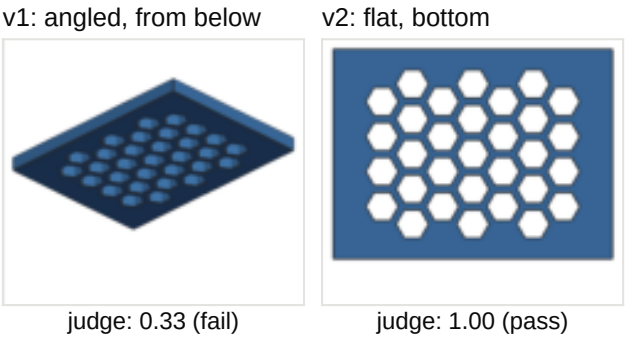


Figure 2. Why the renders include a flat bottom view. The reference solution for a honeycomb panel with hexagonal through-holes, seen from below in our first render version (left) and the current one (right). From the angled view the first judge, gemini-3.1-flash-lite, concluded "the holes are not actual through-holes; they are recessed pockets" and failed this correct part, although the flat top view already showed the background through the holes. With the flat bottom view, and a prompt line saying that only flat views show a through-hole as see-through, it passed. Fig. 4b shows the reverse case: pockets that the flat bottom view exposes.

declining both count, and for the 8 build requests without a fixed size, such as the phone stand in Fig. 4c, any valid mesh counts. The organic suite has no expected sizes, so there a run is correct when it produces a valid mesh.

Size alone does not show whether the part has the requested features. The second metric, *right size + shape*, therefore also requires the visual judge (Sec. 4.4) to pass the part. On the organic suite, any part that came out, valid mesh or not, counts if the judge passes it.

To measure the repair loop, we report the share of runs that are correct after at most k attempts, for $k = 1$ to 4; $k = 1$ is the first-try rate. Retrying a whole request is measured by pass@k over the 3 runs of a case [3]: the chance that at least one of k runs is correct. Cost is measured in tokens (input and output, thinking included) and dollars per correct part, which is everything spent on a builder's runs, failed ones included, divided by its correct runs. Time is the wall-clock time of one build, all attempts included.

4.4. Visual judge

The size check cannot see a missing hole, or a plain disc where a thumbwheel was requested (Fig. 4a). We therefore score each part with a multimodal model as a judge [18]. The part is rendered from six views (Fig. 2; a full render is in Fig. 8 in the appendix): front, right, top and bottom straight on, where a through-hole shows the white background, and two angled views. The judge gets the six views in one image, together with the request text and the part's measured size. It scores the part from 1 to 10 on four bands: matches the request, small deviations, a requested feature wrong or missing, and a different object. The score is rescaled to 0–1, and 0.5 and above passes. The flat bottom view was added after our first judge failed correct parts with through-holes (Fig. 2).

All our earlier studies used Gemini 3.1 Flash Lite as the judge. Before judging this round, we compared it with three stronger models on 204 items, each judged 3 times. 58 items are reference checks: a reference solution judged against its own request, which should pass, or against another case's, which should fail. The other 146 are parts graded by hand on the same four bands, 100 of them graded blind, without seeing the builder, the configuration or any judge's score. The rule, written before the comparison, was to take the judge with the best held-out accuracy, or the cheaper one if its interval overlapped the old judge's. That rule picks Claude Sonnet 5.5 (Table 3), but judging every part with it would have cost about $5.90, more than our remaining budget. We therefore departed from the rule. Gemini 3.8 Flash, statistically tied with Sonnet (92% vs. 96% held-out accuracy, McNemar $p = 0.08$), judges the main suite. On organic parts it is much weaker (71% vs. 91% on 34 graded parts), so Sonnet judges the organic suite. Sec. 5.7 reports the comparison.

4.5. Statistics

Each builder ran each case 3 times, and runs of the same case are not independent: a model that cannot build a part usually fails all three times. We therefore treat the case as the unit. A rate is the mean over cases of each case's rate over its runs, and its 95% interval is a bootstrap over cases [7]. Two builders or configurations are compared on the same cases: we report the mean difference per case with a bootstrap interval, and an exact sign test on the cases

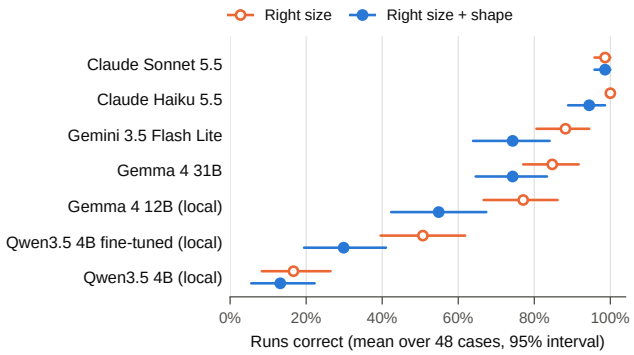


Figure 3. Main suite, per model, with 3 retrieved examples. *Right size*: built at the expected size (or declined when it should be). *Right size + shape*: also passed by the visual judge (gemini-3.8-flash, Sec. 4.4). Mean over the 48 cases of each case's rate over 3 runs; bars are 95% bootstrap intervals over cases.

where one side did better, ties left out. The 15 comparisons were written into the study's manifest before their runs; their p-values are not corrected for multiple tests. For the judges, held-out accuracy judges each item with a pass threshold picked on the other half of the cases. Two judges are compared with McNemar's exact test on the items where they disagree [13], and each judge's agreement with the hand grades is given by Cohen's kappa [5].

5. Results

5.1. Which model builds the part (RQ1)

Fig. 3 shows both metrics for each builder on the main suite, with retrieved examples. Claude Sonnet 5.5 builds a part of the right size and shape in 99% [96–100] of runs, and Claude Haiku 5.5 in 94% [89–99]. Both are well above Gemini 3.5 Flash Lite at 74% [64–84]: Sonnet by 24 points [+14, +35] and Haiku by 20 points [+10, +31] (both $p < 0.001$). Gemma 4 31B matches Gemini Flash Lite at 74%, the local Gemma 4 12B reaches 55%, and the Qwen 4B models 30% (fine-tuned) and 13% (base). The gap between the two metrics is the share of parts with the right size but the wrong shape: none for Sonnet, 14 points for Gemini Flash Lite and 22 points for Gemma 4 12B. With Sonnet at 99%, the main suite can no longer rank the strongest models.

5.2. Retrieved examples (RQ2)

Retrieved examples make no measurable difference for the API models (Table 2). Gemini Flash Lite builds the right part 74% of the time with and without them, Sonnet 99% vs. 98%, and Haiku 94% vs. 95%. They do help these models get it right sooner (first try: Gemini Flash Lite 55% to 67%, Haiku 90% to 97%); without them the repair loop makes up the difference. The fine-tuned Qwen is the only builder

	Right size + shape			First try	K tokens
Builder	RAG/none	Diff. [CI]	p	RAG/none	RAG/none
Sonnet	99/98	+1 [0, 2]	1.00	97/96	2.7/2.1
Haiku	94/95	−1 [−6, 6]	0.69	97/90	3.3/3.3
Gemini FL	74/74	+1 [−8, 9]	0.80	67/55	4.1/3.6
Qwen FT	30/18	**+12** [3, 21]	0.04	28/10	25/27
Qwen	13/10	+3 [−3, 10]	0.51	10/8	85/177

Table 2. Main suite with three retrieved examples (RAG) and without (none), for the five builders run both ways. Only the fine-tuned model gains clearly (RAG better on 15 cases, worse on 5); for the others most cases tie (Sonnet differs on 1 of 48). Percentages, mean over the 48 cases. *Diff.*: paired by case, percentage points, with a 95% bootstrap interval and an exact sign test on the cases that differ (not corrected for the five tests). *First try*: built at the expected size (or declined when it should be) by the first attempt. *K tokens*: thousands of input and output tokens per correct part, failed runs included; compare within a row, as each model family counts tokens differently. Gemini FL: Gemini 3.5 Flash Lite; Qwen: Qwen3.5 4B, local; FT: fine-tuned, trained with examples in every prompt.

with a clear gain: 30% vs. 18% right size and shape (+12 points [+3, +21], $p = 0.04$), and 28% vs. 10% on the first try. It was trained with three examples in every prompt, so this measures the examples together with the format it was trained on. For the base Qwen the gain is within noise (13% vs. 10%, $p = 0.51$), although with examples it produces a valid part more often (12% of runs vs. 3%) and needs half the tokens per correct part.

5.3. Fine-tuning a small local model (RQ3)

Fine-tuning on verified builds triples the small model's right-size rate, from 17% to 51% (+34 points [+22, +47], $p < 0.001$), and more than doubles its right size and shape, from 13% to 30% ($p = 0.007$). It also needs a third of the tokens per correct part (25K vs. 85K). It still falls well short of the local Gemma 4 12B, 25 points lower on right size and shape ($p < 0.001$). On the organic suite it learned to write SDF code that runs (a valid mesh in 36% of runs vs. 3%), but not to sculpt the object: the judge passes 1% of its parts.

5.4. Model size versus running locally

The two Gemma 4 models share a family; the 31B runs on Google's servers, the 12B on our GPU. The 31B is 19 points better on right size and shape (74% vs. 55%, [+6, +32], $p = 0.005$), while its 8-point lead on right size alone is not significant ($p = 0.19$). At 31B the family matches Gemini Flash Lite, so the gap between the local default and the API models comes mostly from model size, not from running locally.

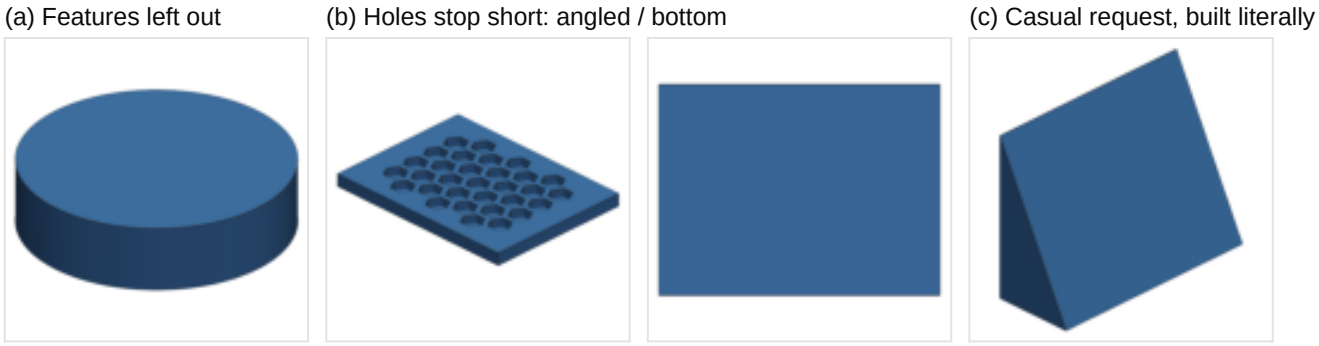


Figure 4. Typical failures: each part is the right size, so the size check accepts it, and the judge fails its shape. (a) Gemini Flash Lite, "a thumbwheel 40 mm in diameter and 10 mm thick with 30 semicircular 2 mm notches around its rim and a hexagonal socket 5 mm across the flats through the centre": a plain disc (score 0.22; the same in all 3 runs). (b) Haiku, a honeycomb panel of "hexagonal through-holes": the holes are pockets and the bottom is solid, visible only from below (0.44; 2 of 3 runs). (c) Haiku, "something to prop my 70 mm wide phone up at an angle on my desk": a plain wedge with no lip to stop the phone sliding (0.44; all 3 runs).

5.5. Repair loop and retries (RQ4)

Fig. 5 shows how many runs are correct when the loop stops after k attempts. Mid-strength models gain the most from repair: Gemini Flash Lite goes from 67% after one attempt to 88% after four, Gemma 4 31B from 52% to 85%, and Gemma 4 12B from 53% to 77%. Sonnet and Haiku, right on the first attempt 97% of the time, have little to repair. Sculpting depends on the loop almost entirely: Gemini Flash Lite's first SDF attempt gives a valid mesh in 1% of runs, and up to four attempts in 82%; Haiku goes from 26% to 91%. The small Qwen models rarely recover: after a script error, the base model's next attempt gave a valid part in 1% of 463 cases. Retrying the whole request adds to the loop: with three tries (pass@3), Gemini Flash Lite reaches 96%, Gemma 4 12B 88% and the fine-tuned Qwen 73%.

5.6. Organic shapes: sculpting or downloading (RQ5)

Sculpting with a strong model beats downloading (Fig. 7). Judged by Sonnet, Haiku's sculpted parts have the right shape in 82% [71–92] of runs and Gemini Flash Lite's in 52% [40–64] (Haiku +30 points [+17, +43], $p = 0.001$). Thingiverse's first matching download is right for 37% [20–53] of cases; Gemini Flash Lite is 16 points higher, but this is not significant ($p = 0.33$). A download came out for 67% of cases, often the wrong object. The small local models cannot sculpt: the judge passes 1% of the fine-tuned Qwen's parts and none of the base model's. The Claude judge does not favour Claude's parts: Gemini 3.8 Flash rates Haiku's parts even higher, at 90%.

5.7. How far the judge can be trusted

Table 3 compares the four judges. The judge of our earlier studies, Gemini 3.1 Flash Lite, is the weakest: 83% held-out accuracy, with 35 of the 204 items wrong, of which each of the other three judges fixes 26 to 29. It failed Claude-built parts 50 points more often than the hand grades (12 graded parts), so earlier studies judged by it understated the strongest models. On mechanical parts the three newer judges are close (92–95%), but on organic parts only Sonnet stays reliable (91%, against 71% for Gemini 3.8 Flash and 65% for Haiku). No judge clearly favours its own model family: for the three newer judges, the gap between the judge's pass rate and the hand grades' stays within 12 points for every family of builders. All four judges are consistent, giving the same verdict in all three runs for 95–99% of items.

	Held-out	Graded parts			Same	
Judge	accuracy	Mech.	Org.	κ	3/3	$/item
Gemini 3.1 FL*	83 [77–88]	77	79	0.54	95	0.0005
Gemini 3.8 Flash†	92 [88–95]	94	71	0.76	**99**	–
Haiku 5.5	90 [85–94]	92	65	0.70	96	**0.0004**
Sonnet 5.5‡	**96** [93–98]	**95**	**91**	**0.87**	97	0.005

Table 3. Four judges on the same 204 items, 3 runs each: 58 reference checks (a reference solution against its own request or another case's) and 146 parts graded by hand. Percentages. *Held-out accuracy*: each item judged with a pass threshold picked on the other half of the cases, with a 95% bootstrap interval over items. *Graded parts*: held-out accuracy on the 112 mechanical and 34 organic hand-graded parts. κ: Cohen's kappa against the hand grades (pass/fail). *Same 3/3*: items given the same verdict in all 3 runs. $/item: API price; Gemini 3.8 Flash has no listed price. FL: Flash Lite. *Judge of all earlier studies. †Judges the main suite. ‡Judges the organic suite.

5.8. Cost and time

Haiku builds the cheapest correct part, at $0.0009, followed by Gemini Flash Lite at $0.0029 and Sonnet at $0.011

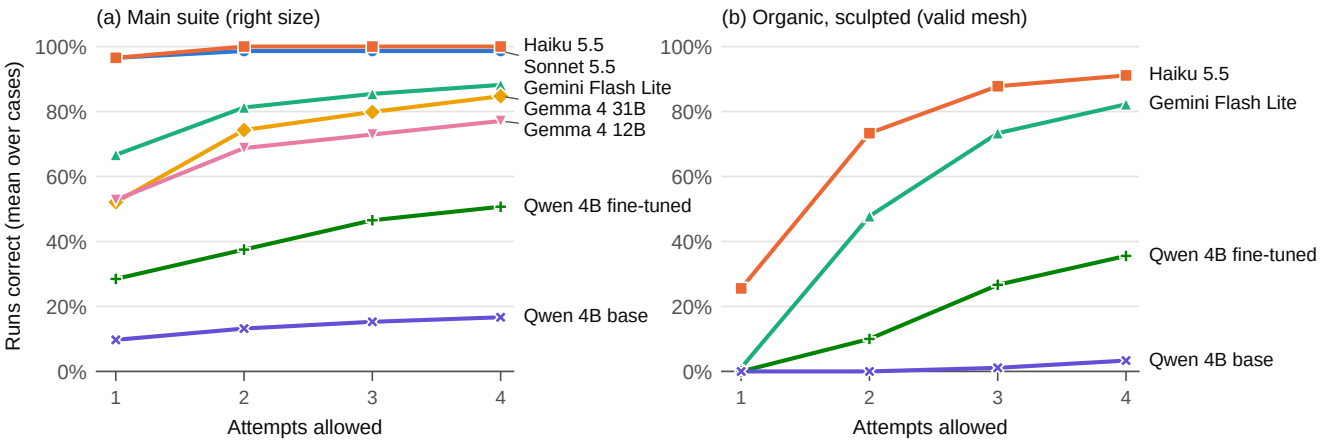


Figure 5. Runs correct when the loop is stopped after k attempts; each failed attempt sends the error back for a repair, and the app allows 4. (a) Main suite with 3 retrieved examples: built at the expected size (or declined when it should be). (b) Organic objects sculpted with SDF code: a valid mesh (shape not judged). Mean over cases of each case's rate over 3 runs. Gemma 4 12B and both Qwen models run locally; the others through APIs. Sonnet's line lies just under Haiku's in (a).

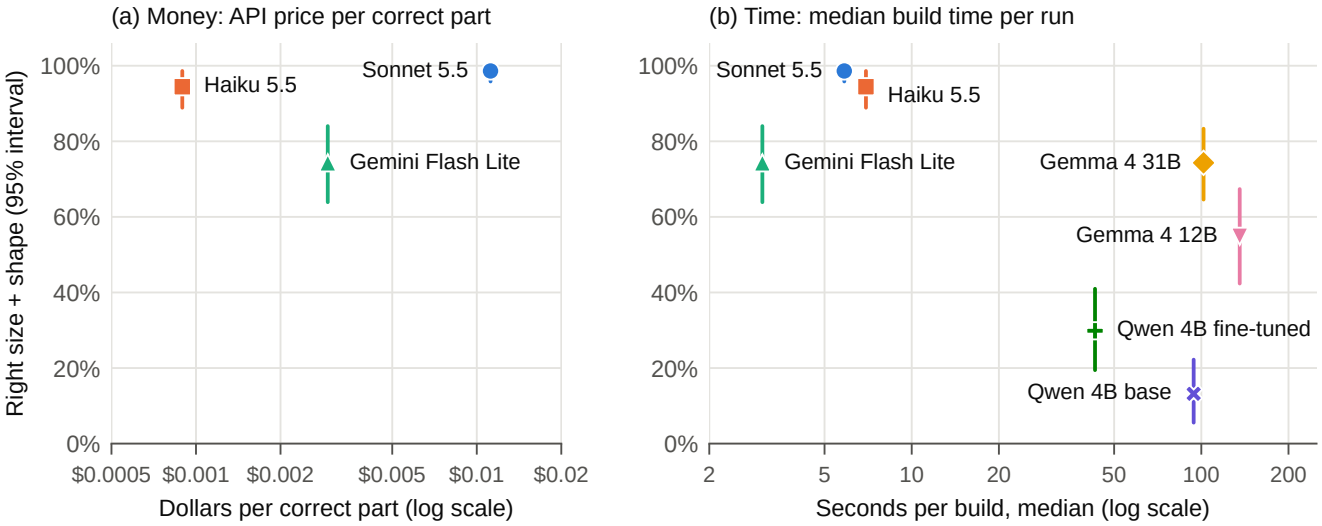


Figure 6. Quality against cost on the main suite (3 retrieved examples). Vertical: right size + shape, mean over cases with 95% intervals (as in Fig. 3). (a) Dollars per correct part: everything spent on a model's runs, failed and repaired ones included, divided by its runs at the right size; only models with a known API price. Gemma 4 31B ran on Google's free tier, and local models cost nothing per request. (b) Median wall-clock time of one build, all attempts included. Gemma 4 12B and both Qwen models run locally on one RTX 3080; Gemma 4 31B is slow because it thinks at length: AI Studio counted 738K output tokens for its runs, 5.7 times what the API reported, at about 35 tokens/s.

(Fig. 6a). Gemini Flash Lite is the fastest, with a median of 3 s per build; Sonnet and Haiku take 6 and 7 s. Local models are much slower: Gemma 4 12B takes a median of 136 s per build (581 s at the 90th percentile), most of it thinking (11.5K output tokens per run), and the fine-tuned Qwen 43 s. The whole round, builds and judging, cost $8.58 at Anthropic and $2.15 at Google (two Google models have no listed price), plus about 36 hours of local GPU time.

Local models cost no money per request, only electricity, which we did not measure. As a rough estimate, assuming the whole machine draws 400 W and electricity costs $0.16/kWh, the GPU time of the main-suite runs comes to about $0.006 per correct part for Gemma 4 12B, $0.002 for the fine-tuned Qwen 4B and $0.013 for the base Qwen 4B, the same range as the API models in Fig. 6a. These figures scale directly with both assumptions, so we leave them out of the figure; build time (Fig. 6b) is the measured cost of running locally.

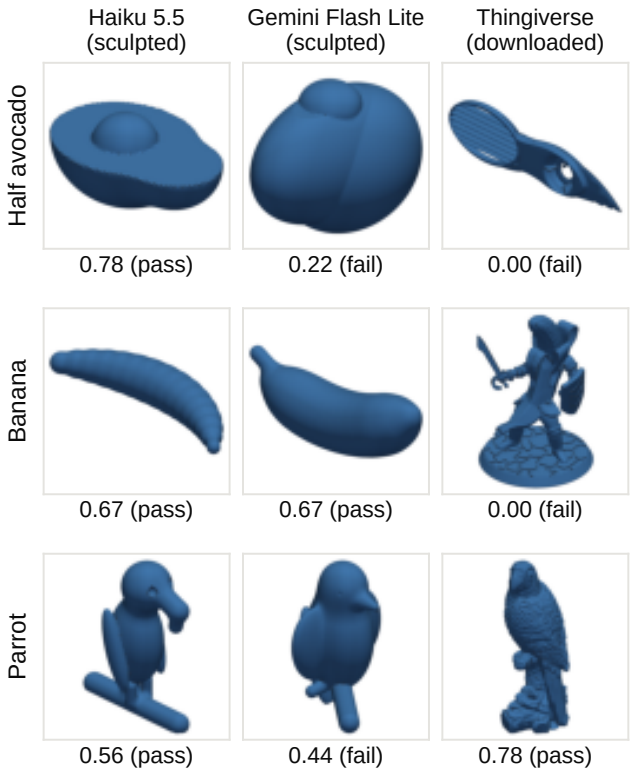


Figure 7. Three organic requests, sculpted with SDF code by two models or downloaded from Thingiverse (the first result whose name or tags hold the object word), in the angled view the judge sees, with the organic judge's score (claude-sonnet-5-5; 0.5 passes). Sculpted parts are each model's median run of 3. Downloads are often the wrong object although their name matches: the "avocado" is an avocado peeler and the "banana" a banana-headed knight; for the parrot, a 3D scan beats both sculpts. Thingiverse models (CC BY-NC / BY-NC-SA): *Avocado Peeler* by ademczuk, *28mm Banana Knight v2* by BigMrTong, *Macaw parrot – 3D Scan* by OpenScan.

6. Discussion

Of the kinds of knowledge in Sec. 1, knowledge in the weights matters most: a larger model or fine-tuning moves right size and shape by 17 to 24 points, more than any other change. Feedback from running the code helps mid-strength models most (21 to 33 points of right size for Gemini Flash Lite and both Gemma models) and makes sculpting possible. Retrieved examples help only models that depend on the format: the fine-tuned model, trained with them, and the base model, which needs them for a valid part; models that know CadQuery do as well without them. For organic objects, a strong model sculpting with code beats an outside repository.

Fig. 4 shows typical failures. For the API models and Gemma 4 12B, the most common failure is a part of the right size with the wrong shape: 20 of 126 runs for Gemini Flash Lite and 30 for Gemma 4 12B. Three kinds recur: a requested feature left out (Fig. 4a), a cut that stops short, such as pockets instead of through-holes (b), and a design that misses the purpose of a loosely worded request (c). The size check cannot catch these; the judge can. The small Qwen models mostly end without a part: 101 of the base model's 126 runs end in a script error that its repairs do not fix. On the organic suite, Thingiverse mostly fails by returning the wrong object, such as an avocado peeler for half an avocado (Fig. 7).

The benchmark is small: 48 main and 30 organic cases, so intervals for mid-strength models span 20 to 25 points. One person graded the 146 parts used to choose the judge, and only 34 of them are organic, so the judges' organic accuracies are uncertain. The main-suite judge departs from our pre-set rule for budget reasons (Sec. 4.4), and the 15 comparisons are not corrected for multiple tests. Token counts are comparable only within a model family (Gemma 4 31B's API under-reports output tokens); local times come from one RTX 3080.

The benchmark calls the builder directly, so the intake gate, the full app path and combined parts are not measured, and Gemma 4 12B and Sonnet were not run on the organic suite.

7. Conclusion

We built an agent that turns a request into an STL file by writing CadQuery or SDF code, running it, checking the mesh and repairing the code, and measured which kinds of knowledge help which models on 78 requests, 3 runs each. Strong API models are near the ceiling of our main suite (Sonnet 99%), and the local default's gap is mostly one of size: a 31B model of the same family closes it (RQ1). Retrieved examples help only models that depend on the format, such as our fine-tuned model (RQ2). Fine-tuning a 4B model on verified builds triples its right-size rate, but it stays behind the 12B model (RQ3). The repair loop adds 21 to 33 points for mid-strength models and makes sculpting possible, and retrying the whole request adds more (RQ4). For organic objects, sculpting with a strong model beats downloading the first match (RQ5).

Future work: a harder benchmark to separate the strongest models, a second hand grader, an evaluation of the intake gate and the full app, and running the judge inside the build loop to repair wrong shapes like wrong sizes.

AI usage statement

The codebase for this project was built primarily with the assistance of Claude Code, Anthropic's AI coding agent. For this report, Claude Code was used for formatting, grammatical fixes and the outline. Separately, Claude models are among the builders and judges evaluated in Sec. 4.

References

[1] Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. Generating CAD code with vision-language models for 3D designs. In *ICLR*, 2025. 2

[2] CadQuery contributors. CadQuery: A python parametric CAD scripting framework. https://github.com/CadQuery/cadquery, 2024. 2

[3] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. 4

[4] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *ICLR*, 2024. 2

[5] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. 5

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *NeurIPS*, 2023. 2

[7] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993. 4

[8] Michael Fogleman. sdf: Simple SDF mesh generation in Python. https://github.com/fogleman/sdf, 2021. 3

[9] Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts. In *NeurIPS*, 2024. 1

[10] Emmett Lalish et al. Manifold: Geometry library for topological robustness. https://github.com/elalish/manifold, 2024. 2

[11] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020. 2

[12] Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023. 2

[13] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. 5

[14] Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. CAD-Recode: Reverse engineering CAD code from point clouds. In *ICCV*, 2025. 2

[15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023. 2

[16] UltiMaker. Thingiverse. https://www.thingiverse.com, 2026. Accessed November 2026. 2

[17] Haoyang Xie and Feng Ju. Text-to-CadQuery: A new paradigm for CAD generation with scalable large model capabilities. *arXiv preprint arXiv:2505.06507*, 2025. 1

[18] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *NeurIPS Datasets and Benchmarks Track*, 2023. 2, 4

A. Appendix

A.1. Benchmark cases

Every request in both suites, as sent to the builder. For the main suite, each case's category, whether it should be built or declined (*either*: both count), and the expected size, which is checked in any axis order. For the organic suite, the phrase used for the Thingiverse search. The files are generated by `python -m scripts.report_figures appendix` from `eval/cases.json` and `eval/organic_cases.json`.

Main suite (48 cases)

s01 *simple, build, 12 × 12 × 15 mm*: A cylindrical spacer with 12mm outer diameter, 6.5mm inner diameter, 15mm long.
s02 *simple, build, 25 × 25 × 25 mm*: A 25mm cube with a 10mm diameter hole drilled through the centre from top to bottom.
s03 *simple, build, 34.64 × 30 × 8 mm*: A hexagonal prism 30mm across the flats and 8mm tall.
s04 *simple, build, 70 × 20 × 4 mm*: A 70mm x 20mm x 4mm bar with a 40mm long, 6mm wide slot through it, centred along its length.
s05 *simple, build, 40 × 40 × 30 mm*: A solid cone 40mm in base diameter and 30mm tall.
m01 *medium, build, 120 × 80 × 4 mm*: A 120mm x 80mm x 4mm mounting plate with a 3 by 2 grid of 5mm holes spaced 30mm apart, centred on the plate, and all four corners rounded with an 8mm radius.
m02 *medium, build, 100 × 60 × 15 mm*: An open-top rectangular tray 100mm x 60mm x 15mm with 2mm walls and a 2mm floor, with one 2mm divider wall splitting it into two equal compartments along its length.
m03 *medium, build, 30 × 30 × 15 mm*: A round knob 30mm in diameter and 15mm tall, with a 6mm D-shaped shaft hole 10mm deep in the bottom; the flat of the D is 1mm in from the circle.
m04 *medium, build, 60 × 60 × 5 mm*: A 60mm diameter disc, 5mm thick, with a 20mm centre hole and eight 4mm holes evenly spaced on a 45mm bolt circle.
m05 *medium, build, 80 × 20 × 5 mm*: A cable comb: an 80mm x 20mm x 5mm bar with six 4mm wide, 12mm deep slots cut into one long edge, evenly spaced along it.
m06 *medium, build, 100 × 30 × 20 mm*: A U-channel 100mm long, 30mm wide and 20mm tall, with 3mm thick walls and base, open at the top and both ends.
m07 *medium, build, 50 × 50 × 60 mm*: A cylindrical cup 50mm outer diameter and 60mm tall with 2mm walls and a 2mm base, open at the top, with the outer bottom edge filleted with a 3mm radius.
m08 *medium, build, 40 × 40 × 10 mm*: A pipe clamp ring: 10mm tall, 32mm inner diameter, 4mm wall, with a 2mm wide gap cut through one side of the ring.
m09 *medium, build, 50 × 50 × 6 mm*: A 50mm x 50mm x 6mm plate with a 30mm square pocket, 3mm deep, centred in the top face; the pocket's four inside corners are rounded with a 4mm radius.
m10 *medium, build, 20 × 20 × 40 mm*: A 20mm diameter pin 40mm long with a 2mm chamfer on both end edges and a 3mm wide, 1.5mm deep groove around it halfway along its length.
m11 *medium, build, 25 × 40 × 3 mm*: A 25mm wide, 40mm tall, 3mm thick hanging plate with a keyhole cut through it: an 8mm diameter hole centred 12mm above the bottom edge, with a 4mm wide slot running 10mm straight up from the hole's centre.
m12 *medium, build, 25 × 25 × 10 mm*: A shaft collar: 25mm outer diameter, 10mm bore, 10mm long, with a 3.3mm radial hole drilled from the outside into the bore, centred along its length.
m13 *medium, build, 16 × 16 × 60 mm*: A stepped shaft on one axis: 10mm diameter for the first 20mm, then 16mm diameter for 15mm, then 8mm diameter for the last 25mm.
m14 *medium, build, 70 × 50 × 25 mm*: An open-top box 60mm x 40mm x 25mm with 2mm walls and floor, with a 3mm thick flange sticking out 5mm all around the outside of the top rim, flush with the top.
m15 *medium, build, 15.01 × 13 × 35 mm*: A bolt blank: a hexagonal head 13mm across the flats and 5mm tall, with a plain 8mm diameter shank 30mm long (no thread).
m16 *medium, build, 57.06 × 54.27 × 6 mm*: A five-pointed star 6mm thick: the points lie on a 30mm radius circle and the inner corners on a 12mm radius circle.
m17 *medium, build, 100 × 20 × 40 mm*: A 100mm long, 20mm deep, 40mm tall block with a 60mm diameter semicircular arch cut through it from front to back, centred on the bottom edge.
h01 *hard, build, 44 × 44 × 8 mm*: A spur gear with 20 teeth, module 2, 8mm thick, with a 5mm centre bore.
h02 *hard, build, no fixed size*: A phone stand: a 70mm wide, 80mm deep, 5mm thick base plate; a 5mm thick, 90mm long back support leaning back at 65 degrees from the base; and a 10mm tall lip along the front edge to stop the phone sliding.
h03 *hard, build, no fixed size*: A wall hook: a 40mm x 60mm x 5mm back plate with two 4mm countersunk screw holes, and a curved hook arm that sticks out 35mm from the lower part of the plate and curves upward at the end.
h04 *hard, build, 40 × 40 × 12 mm*: A pulley wheel 40mm in diameter and 12mm wide with an 8mm bore and a V-groove around its rim, 6mm wide at the surface and 4mm deep.
h05 *hard, build, 20 × 20 × 32 mm*: A coil spring made from 2mm diameter round wire: 20mm outer diameter, 5 turns at a 6mm pitch.
h06 *hard, build, 60 × 60 × 23 mm*: A desk cable grommet: a 60mm diameter, 3mm thick round flange on top of a 50mm outer diameter, 20mm long tube with 2mm walls, open through the middle, with a 15mm wide slot cut through the flange and tube from one side to the centre so a cable can be slipped in.
h07 *hard, build, 40 × 40 × 20 mm*: A 90 degree pipe elbow: a tube with 20mm outer and 16mm inner diameter, bent through 90 degrees around a 30mm centreline radius, with no straight ends.
h08 *hard, build, 80 × 60 × 5 mm*: An 80mm x 60mm x 5mm honeycomb panel: a grid of hexagonal through-holes 8mm across the flats with 2mm walls between them, leaving a solid border at least 3mm wide around the edge.
h09 *hard, build, 25 × 25 × 90 mm*: A tool handle 25mm in diameter and 90mm long overall, with a fully rounded hemispherical back end and six 4mm diameter half-round flutes running along the straight part, evenly spaced around it.
h10 *hard, build, 60 × 50 × 40 mm*: An L-bracket 50mm wide with a 60mm long horizontal leg and a 40mm tall vertical leg, both

5mm thick, braced by two 5mm thick triangular gussets flush with the side edges, with two 5mm holes in each leg.

h11 *hard, build, 80 × 80 × 70 mm*: A tapered square planter 70mm tall, 60mm square at the bottom and 80mm square at the top, with 3mm walls and base, open at the top, and a 10mm drainage hole in the centre of the base.

h12 *hard, build, 40 × 40 × 10 mm*: A thumbwheel 40mm in diameter and 10mm thick with 30 semicircular 2mm diameter notches evenly spaced around its rim and a hexagonal socket 5mm across the flats through the centre.

h13 *hard, build, 60 × 40 × 30 mm*: An open-top box 60mm x 40mm x 30mm with 2mm walls and floor, with a 1.5mm tall, 1mm deep groove cut into the inside of both long walls, 3mm below the top edge, for a sliding lid.

p01 *casual, build, 150 × 25 × 3 mm*: Make me a ruler with my name engraved on it. The ruler is 150mm long, 25mm wide and 3mm thick, with the name ALEX engraved 0.5mm deep and a tick mark every 10mm along one long edge.

p02 *casual, either, no fixed size*: Make a die that has 2 snakes and 4 ladders instead of numbers.

p03 *casual, decline*: Create a tree that has a cat on it.

p04 *casual, decline*: A realistic, anatomically accurate model of a human hand.

p05 *casual, either, no fixed size*: A chess knight.

p06 *casual, build, 25 × 25 × 40 mm*: A chess rook 40mm tall and 25mm across at its widest point (the base), with a castle top that has four notches cut into its rim.

p07 *casual, decline*: A bust of Abraham Lincoln.

p08 *casual, either, no fixed size*: A low-poly fox figurine about 60mm tall.

c01 *casual, build, 80 × 40 × 25 mm*: Can you make me something to stand my AA batteries up in? 8 of them in two rows of four, holes 15mm wide and 20mm deep, in an 80 x 40 x 25 block.

c02 *casual, build, 50 × 30 × 20 mm*: I need a little open tray to keep SD cards in. It has to fit a 50 x 30 x 20mm spot on my shelf, thin walls are fine.

c03 *casual, build, no fixed size*: something to prop my 70mm wide phone up at an angle on my desk

c04 *casual, build, no fixed size*: My table leg is a 25mm round tube and it scratches the floor. Make a cap that slides 15mm onto the end, with a 3mm thick bottom.

c05 *casual, build, no fixed size*: a hook that screws to the underside of my desk so I can hang my headphones on it

Organic suite (30 cases)

o01 *search: sitting cat*: A sitting cat about 50 mm tall, tail curled around its front paws, ears pointing up.

o02 *search: horse figurine*: A standing horse figurine, 60 mm tall, four legs, long neck and head raised, a tail hanging down.

o03 *search: frog*: A frog sitting on its folded back legs, about 40 mm long, with big bulging eyes on top of its head.

o04 *search: tulip flower*: A tulip flower 80 mm tall: a cup-shaped bloom of petals on a straight stem with two long leaves, on a small round base.

o05 *search: octopus*: A cartoon octopus 50 mm tall: a round head and eight curling tentacles spread around it on the ground.

o06 *search: owl figurine*: An owl figurine 50 mm tall: an egg-shaped body, a round face with two large eyes and a small beak, ear tufts, wings folded at the sides.

o07 *search: rubber duck*: A rubber duck, 45 mm tall, with a flat bottom so it stands.

o08 *search: brontosaurus*: A long-necked dinosaur (brontosaurus) 80 mm long: a heavy body on four pillar legs, a long neck with a small head, and a long tail.

o09 *search: hand sculpture*: A human hand, palm open with fingers spread, standing upright on its wrist, 90 mm tall.

o10 *search: apple fruit*: An apple 50 mm wide with a short stem and one leaf.

o11 *search: dolphin*: A dolphin leaping in an arc, 70 mm long, with a dorsal fin, flippers and a tail fluke, the tail touching a small round base.

o12 *search: turtle*: A turtle 60 mm long: a domed shell, four stubby legs, a head poking out the front and a short tail.

o13 *search: seahorse*: A seahorse 70 mm tall, upright, with a curled tail touching a small round base, a long snout and a ridged belly.

o14 *search: crab*: A crab 60 mm wide: a flat oval shell, two raised claws at the front, four legs on each side, and eyes on short stalks.

o15 *search: starfish*: A starfish 70 mm across with five thick arms tapering to rounded tips, lying flat, with a bumpy top.

o16 *search: shark figurine*: A shark 80 mm long, swimming, on a small stand: a pointed snout, a tall dorsal fin, two side fins and a forked tail.

o17 *search: walrus*: A walrus 55 mm tall, resting on its belly with its front flippers on the ground, a round head, whiskered snout and two long tusks pointing down.

o18 *search: chicken figurine*: A chicken 50 mm tall standing on two legs, a round body, a comb on top of its head, a small beak and tail feathers raised at the back.

o19 *search: llama*: A standing llama 75 mm tall: four thin legs, a long upright neck, a small head with tall pointed ears and a fluffy body.

o20 *search: parrot figurine*: A parrot 60 mm tall perched upright on a short branch, with a curved hooked beak, folded wings and a long tail hanging down.

o21 *search: pineapple*: A pineapple 80 mm tall: an oval fruit with a criss-cross scaled surface and a crown of spiky leaves on top.

o22 *search: banana*: A single banana 90 mm long, curved, with a short stem at one end, lying on its side.

o23 *search: strawberry*: A strawberry 40 mm tall, heart-shaped, with small seed dimples over its surface and a crown of leaves on top.

o24 *search: avocado half*: Half an avocado 70 mm long, cut side up, lying flat, with the round pit sitting in the middle.

o25 *search: garlic bulb*: A garlic bulb 45 mm tall: plump cloves bulging around a rounded base, gathered into a pointed tip at the top.

o26 *search: rose flower*: A single rose 90 mm tall: a tight spiral bloom of petals on a straight stem with two leaves, on a small round base.

o27 *search: palm tree*: A palm tree 90 mm tall: a slightly curved ringed trunk on a small round base, with long drooping fronds spreading from the top.

o28 *search: molar tooth*: A molar tooth 40 mm tall, standing on its two roots, with a wide crown and a bumpy top.
o29 *search: croissant*: A croissant 80 mm long, crescent-shaped, with rolled layers that swell in the middle and taper to the two curved ends.
o30 *search: cupcake*: A cupcake 50 mm tall: a ridged paper cup, a swirl of frosting on top and a cherry at the peak.

A.2. Prompts

The system prompts used in the round: for CadQuery builds, for SDF sculpting, and for the visual judge (version 2, paired with the render with a flat bottom view). The request, and with RAG the retrieved examples, go in the first user message after it.

```
You are a CAD engineer that turns a plain-English part
     description into a CadQuery (Python) script.
The script is executed, exported to STL, and checked
     automatically. Units are millimetres.

OUTPUT FORMAT (follow exactly)
PLAN:
- 2-6 short bullets: base shape, features in order, key
     dimensions, any assumption you made.
BBOX: X, Y, Z
```python
<complete script>
```
BBOX is the overall bounding-box size of the FINISHED
     part in mm (three numbers, any axis order).

If the request cannot reasonably be built from CAD
     primitives and operations (organic or artistic
shapes, characters, logos, anything needing a mesh
     sculpt) reply with ONE line instead:
UNSUPPORTED: <short reason>
If a request is merely underspecified, pick sensible
     dimensions, state them in PLAN, and proceed.

SCRIPT RULES
- Only `import cadquery as cq` (and `math` if needed).
     No file I/O, no other imports.
- Put every dimension in a named variable at the top (mm
     ); derive related values from them.
- The final solid must be passed to `show_object(result)
     ` and be a single connected solid.
- Prefer the fluent API: Workplane(...).box / circle /
     rect / polyline -> extrude / cutBlind / hole.
- Select faces/edges with string selectors: faces(">Z"),
      edges("|Z"), edges("#Z"), faces("<X").
  After faces(">Z") call .workplane() before placing
     holes/sketches on it.
- Holes: .hole(d) / .cboreHole(d, cbore_d, cbore_depth)
     / .cskHole(d, csk_d, csk_angle);
  place several with .pushPoints([...]), .rarray(xs, ys,
      nx, ny), or .polarArray(radius, 0, 360, n).
- Hollowing: .faces(">Z").shell(-wall) (negative =
     inward, selected face is opened).
- Fillet/chamfer radius must be clearly smaller than the
      thinnest adjacent feature, and apply it
  to the whole solid BEFORE cutting small holes next to
     those edges when possible.
- Centering: box() and rect() are centred on the
     workplane origin unless centered=(False, ...).
  extrude() has NO `centered` argument; use extrude(h,
     both=True) to extrude symmetrically.
- Do NOT invent methods (there is no fillet_all, no
     add_hole, no Workplane.sphere with 3 args).
  Methods that exist include: box, sphere, cylinder,
     circle, rect, polygon, ellipse, slot2D,
  extrude, revolve, loft, sweep, union, cut, intersect,
     translate, rotate, mirror, fillet, chamfer.

Learn the style from the examples, but adapt geometry
     and dimensions to the new request rather
than copying. When fixing an error, change only what is
     needed and reply in the same format.
```

```
You are a 3D modeller who turns a plain-English
     description of an organic or sculpted object (an
     animal,
a character, a plant, food, a figurine) into a Python
     script that builds it from signed distance
functions. The script is executed, meshed to STL, and
     checked automatically. Units are millimetres.

OUTPUT FORMAT (follow exactly)
PLAN:
- 2-6 short bullets: the main volumes (body, head, limbs
     ...) with sizes and positions, the pose, any
  assumption you made.
BBOX: X, Y, Z
```python
<complete script>
```
BBOX is the overall bounding-box size of the FINISHED
     part in mm (three numbers, any axis order).

If the request is not a solid object at all (a picture,
     a logo, an effect), reply with ONE line instead:
UNSUPPORTED: <short reason>
If it is merely underspecified, pick a sensible size (
     most figurines 40-80 mm tall), state it in PLAN,
and proceed.

THE LIBRARY (everything there is; points and sizes are (
     x, y, z) tuples in mm)
Shapes:
  sphere(radius, center=(0, 0, 0))
  ellipsoid((rx, ry, rz))   radii, centred
     on the origin
  box(size, center=(0, 0, 0))   full size: a
     number or (x, y, z)
  rounded_box((x, y, z), radius)   centred on the
     origin, edges rounded
  capsule(a, b, radius)   rod with round
     ends from point a to point b
  capped_cylinder(a, b, radius)   flat ends
  capped_cone(a, b, ra, rb)   radius ra at a,
     rb at b (rb near 0 for a point)
  rounded_cone(r1, r2, h)   along +Z from
     the origin, ball ends of radius r1 and r2
  torus(r1, r2)   ring in the XY
     plane: r1 to the tube's middle, r2 the tube
Combining:
  a | b   join   a - b   cut b out of a
     a & b   keep only the overlap
  union(a, b, c, ...)   join many; union(..., k=r)
     blends them all
  .k(r) on the shape being joined or cut blends it in
     with a fillet of about r mm: body | head.k(5).
  Blend body parts together; leave blends off where an
     edge should stay crisp.
Transforms (each returns a new shape):
  .translate((x, y, z))   .rotate(angle, axis) angle in
      RADIANS (math.radians), axis X, Y, Z or a vector
  .orient(axis) turn the shape's +Z toward axis   .
     scale(s) or .scale((sx, sy, sz))
  .elongate((x, y, z)) stretch through the middle   .
     shell(t) hollow skin t thick
  .dilate(r) grow by r
There is no text, no sweep along a curve, no mesh import
     , no surface noise.

SCRIPT RULES
- Start with `from sdf import *`. Only `math` and `numpy
      as np` may also be imported. No file I/O.
```

```
- Z is up. Build the object wherever is convenient.
    After the script, the finished shape is stood on
  z = 0 automatically: moved so its lowest point is 1.5
    mm below z = 0 and cut flat there, a small
  footprint to print on. Never cut at z = 0 yourself: a
    shape built around the origin would lose its
  lower half.
- Put the main sizes in named variables at the top (mm).
     Build the large volumes first, then attached
  features, then cuts (eyes, mouth) last.
- Left-right pairs (ears, arms, legs, eyes): write a
    function of the side, `def side(s):` with s = 1 or
    -1,
  multiplying each y position (and each tilt about X) by
     s, then join side(1) and side(-1).
- Everything must touch: the final shape must be one
    connected solid. Overlap joined parts by a few mm.
- Features thinner than 2 mm will not print; make tails,
     ears, legs and stems at least 2-3 mm thick.
- Pass the final shape to `show_object(result)`.

Learn the style from the examples, but adapt the shapes
    and sizes to the new request rather than copying.
When fixing an error, change only what is needed and
    reply in the same format.
```

```
You check whether a 3D-printable part matches the
    request it was made from.

You see one image with six views of the same part,
    labelled in each corner:
- front, right, top and bottom are FLAT views, looking
    straight at the part. A hole that goes all the way
  through shows the white background through it; a blind
     hole or pocket shows a shaded floor instead.
- "iso front (angled)" and "iso back (angled)" are 3D
    views from above at an angle, for the overall shape
  and the back side. In angled views you look into holes
     at a slant, so even a through-hole looks closed:
  decide whether holes go through from the flat views,
    never from the angled ones.
Each view is zoomed to fit, so sizes can't be compared
    between views, and there is no scale; the measured
overall size is given in the message.

Judge the SHAPE: are the requested features there, in
    the right number, kind and place (holes, slots,
pockets, walls, fillets, text, angles, which side is
    open)? Is anything major missing or added?
Use the measured size for overall dimensions; don't
    guess small dimensions from the picture.
Ignore color, shading, render quality and how the part
    is oriented in the views.

Score 1-10:
  9-10  matches the request
  7-8   matches with small deviations (a slightly
    different proportion or spacing)
  4-6   recognisably the requested part, but a requested
     feature is wrong or missing
  1-3   a different object, or most requested features
    are missing

Reply with only a JSON object, problems before the score
    :
{"matches": ["requested features you can see"], "
    problems": ["missing, wrong or extra features"], "
    score": 1-10}
```

A.3. How the main-suite runs ended

Table 4 counts how each builder's runs of the 42 cases that should be built ended (3 runs each, 126 per builder, with RAG).

Builder	Right part	Wrong shape	Wrong size	No part	
				Script	Other
Sonnet 5.5	124	0	2	0	0
Haiku 5.5	118	8	0	0	0
Gemini 3.5 FL	89	20	3	6	8
Gemma 4 31B	102	11	0	8	5
Gemma 4 12B	65	30	1	17	13
Qwen 4B FT	38	29	10	28	21
Qwen 4B	9	5	3	101	8

Table 4. How the runs of the 42 build cases ended. *Right part*: right size and passed by the judge. *Wrong shape*: right size, failed by the judge. *No part*: the last attempt failed with a script error, or otherwise (the mesh check failed, no code, or the reply was cut off). FL: Flash Lite; FT: fine-tuned.

A.4. What the judge sees

Fig. 8 shows a full render as the judge receives it, for a correct part and for a part of the right size without its holes.

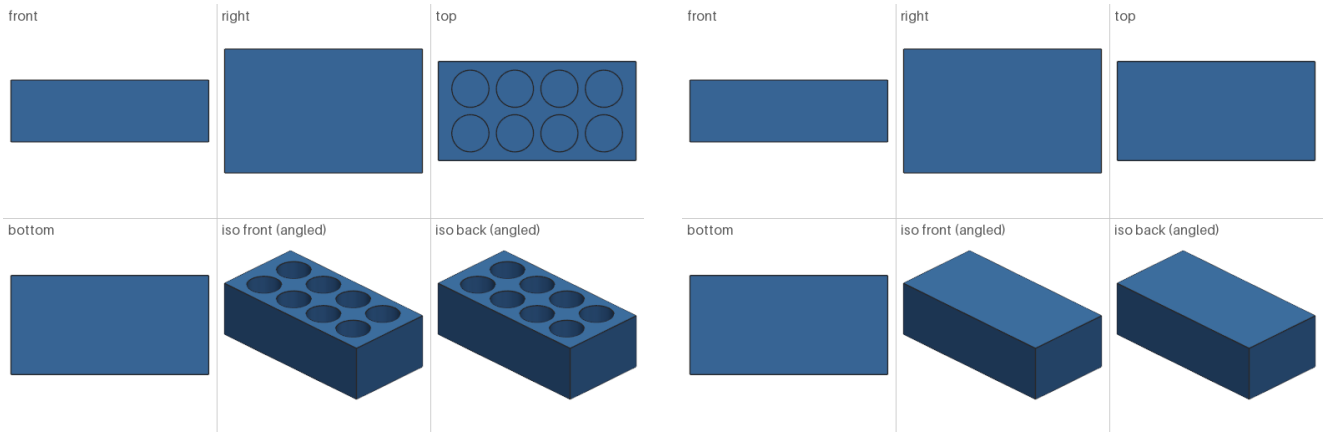
A.5. Renders

The renders of every judged part are kept in `eval/renders/`, and `judge sheet --study report-v1` builds a page that shows them side by side with the judge's score and reasons.

A.6. Spend and hardware

The round cost $8.58 at Anthropic: $3.80 for builds (Sonnet $3.21, Haiku $0.59) and $4.78 for judging (Sonnet $4.55, Haiku $0.22, checks $0.01). At Google, Gemini 3.5 Flash Lite's builds cost $1.36 and judging with Gemini 3.1 Flash Lite $0.79. Gemini 3.8 Flash (judging, 2.5M input and 114K output tokens) and Gemma 4 31B (builds) have no listed price. Local models ran in Ollama with a 16K-token context on one NVIDIA RTX 3080 (10 GB), for about 36 hours of GPU time.

"Can you make me something to stand my AA batteries up in? 8 of them in two rows of four, holes 15mm wide and 20mm deep, in an 80 x 40 x 25 block."



(a) Reference solution: score 1.0 (*pass*).

(b) Gemma 4 31B build: score 0.11 (*fail*). "Missing all 8 holes for the AA batteries; part is a completely solid block."

Figure 8. What the visual judge sees: six views of the part (four flat, two angled), sent in one request with the request text and the measured size. Both parts measure $80 \times 40 \times 25$ mm, so the size check accepts both; only the top and angled views show that (b) has no holes. The judge scores 1–10 on four bands (match, small deviations, a feature wrong, a different object), rescaled to 0–1; 0.5 and above passes. It gave each part the same score in all 3 runs, and the hand grade for (b) is "different object".